PEMROGRAMAN BERORIENTASI OBJEK 2026

Prinsip SOLID

Lima prinsip desain agar kelas mudah diubah, lalu membongkar kasir serba bisa milik KedaiKita

PERTEMUAN

13

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Mengenali tanda kode yang sulit diubah (***code smell***), misalnya kelas serba bisa dan rantai percabangan yang panjang.
2. Menjelaskan kelima prinsip **SOLID** (SRP, OCP, LSP, ISP, DIP) dengan analogi sehari-hari dan contoh kode.
3. Menemukan pelanggaran prinsip SOLID pada sebuah rancangan lalu mengusulkan perbaikannya.
4. Menerapkan ***dependency injection*** lewat konstruktor agar sebuah kelas bergantung pada abstraksi.
5. Melakukan ***refactoring*** dan membuktikan perilaku program tidak berubah dengan `diff` atau `fc`.
6. Menimbang kapan prinsip desain perlu diterapkan, termasuk prinsip pendamping DRY, KISS, dan YAGNI.

Mengingat Kembali



Dari pertemuan sebelumnya

- Pertemuan 09: polimorfisme membuat kita bisa menambah bangun datar baru **tanpa mengubah** fungsi pencetak laporan.
- Pertemuan 09 juga: objek anak harus bisa dipakai di mana pun objek induk diharapkan, tanpa membuat pemanggil kaget.
- Pertemuan 10: kelas abstrak dan *interface* menjadi **kontrak**, alat utama dalam SOLID.
- Pertemuan 12: `List<Menu> daftar = new ArrayList<>()` berarti bergantung pada kontrak, bukan pada wujud.

Agenda Hari Ini

1. Program benar, tetapi sulit diubah
2. S: satu kelas, satu alasan untuk berubah
3. O dan L: menambah perilaku tanpa kejutan
4. I dan D: kontrak kecil dan abstraksi
5. *Refactoring* dan prinsip pendamping
6. Studi kasus KedaiKita: membongkar `KasirSerbaBisa`

BAGIAN 1

1

Program Benar, tetapi Sulit Diubah

Kasir KedaiKita berjalan benar, tetapi Andi takut mengubahnya. Mengapa?

Code Smell: Tanda Kode Mulai Bermasalah

Code smell	Cirinya	Prinsip yang biasanya dilanggar
Kelas serba bisa (*god class*)	satu kelas mengurus banyak hal berbeda, *method*-nya sangat panjang	SRP
Rantai percabangan "jenis"	`if jenis == "a" ... elif ...` yang bertambah setiap ada jenis baru	OCP
Override yang menolak	*method* kelas anak hanya berisi "maaf, tidak bisa" (melempar *exception*)	LSP, ISP
Interface gemuk	kelas dipaksa menulis *method* yang tidak ia butuhkan	ISP
Membuat alat sendiri	menciptakan sendiri objek konkret yang dipakai, sulit diganti atau diuji	DIP
Kode kembar	rumus yang sama disalin ke beberapa tempat	DRY

Kaku (*rigid*): satu perubahan kecil memaksa banyak bagian lain ikut diubah. **Rapuh** (*fragile*): mengubah satu tempat merusak tempat lain.

SOLID dalam Satu Tabel

Prinsip	Arti singkat	Pertanyaan untuk menguji kodemu
S: *Single Responsibility Principle* (SRP)	satu kelas, satu alasan untuk berubah	Adakah lebih dari satu pihak atau aturan yang bisa memaksanya berubah?
O: *Open/Closed Principle* (OCP)	terbuka untuk ditambah, tertutup untuk diubah	Bisakah fitur baru ditambahkan tanpa membongkar kelas lama?
L: *Liskov Substitution Principle* (LSP)	kelas anak bisa menggantikan induknya tanpa kejutan	Apakah kelas anak menepati semua janji kelas induknya?
I: *Interface Segregation Principle* (ISP)	*interface* kecil yang fokus	Adakah kelas yang dipaksa menulis *method* yang tidak ia butuhkan?
D: *Dependency Inversion Principle* (DIP)	bergantung pada abstraksi, bukan pada kelas konkret	Apakah kelas pengatur ini terikat langsung pada kelas konkret?

Dirangkum dan dipopulerkan oleh Robert C. Martin. Kelimanya saling mendukung: satu perbaikan sering memenuhi dua prinsip sekaligus.

BAGIAN 2

2

S: Satu Kelas, Satu Alasan untuk Berubah

Prinsip pertama: pisahkan urusan yang perubahannya datang dari sumber berbeda.

SRP: Siapa yang Akan Meminta Perubahan?

SRP (*Single Responsibility Principle*)

Sebuah kelas sebaiknya hanya punya **satu alasan untuk berubah**, yaitu hanya mengurus satu urusan yang perubahannya datang dari satu sumber yang sama.

- `LaporanSerbaBisa` menghitung nilai **dan** mencetak laporan: dua sumber perubahan, dua alasan untuk berubah.
- Trik cepat: jelaskan tugas kelasmu dalam satu kalimat. Jika penuh kata “dan”, tugasnya kemungkinan terlalu banyak.
- SRP **bukan** berarti satu kelas hanya boleh punya satu *method*.

SRP dalam Kode: Menghitung dan Mencetak Dipisah

`LaporanSerbaBisa` dipecah dua: `PenghitungNilai` hanya menghitung, `PencetakLaporan` hanya mencetak.

Python python/contoh/contoh01_srp.py

```python
class PenghitungNilai:
    def rata_rata(self, daftar: list[int]) -> int:
        return sum(daftar) // len(daftar)

    def predikat(self, rata: int) -> str:
        # ... (dilewati)

class PencetakLaporan:
    def __init__(self, penghitung: PenghitungNilai):
        self._penghitung = penghitung
# ... di dalam main()
penghitung = PenghitungNilai()
PencetakLaporan(penghitung).cetak(data_nilai)
```

Java java/contoh/Contoh01Srp.java

```java
class PenghitungNilai {
    public int rataRata(List<Integer> daftar) {
        // ... (dilewati)
    }
    public String predikat(int rata) {
        // ... (dilewati)
    }
}
class PencetakLaporan {
    private PenghitungNilai penghitung;
    public PencetakLaporan(PenghitungNilai penghitung) {
        this.penghitung = penghitung;
    }
    // ... (dilewati)
}
// ... di dalam main
PenghitungNilai penghitung = new PenghitungNilai();
new PencetakLaporan(penghitung).cetak(dataNilai);
```

BAGIAN 3

3

O dan L: Menambah Perilaku tanpa Kejutan

Jenis baru ditambahkan tanpa membongkar kode lama, dan kelas anak selalu menepati janji induknya.

O: Terbuka untuk Ditambah, Tertutup untuk Diubah

Setiap layanan kirim menjadi kelas yang memenuhi kontrak `LayananKirim`. Layanan baru cukup ditulis sebagai kelas baru.

Python python/contoh/contoh02_ocp.py

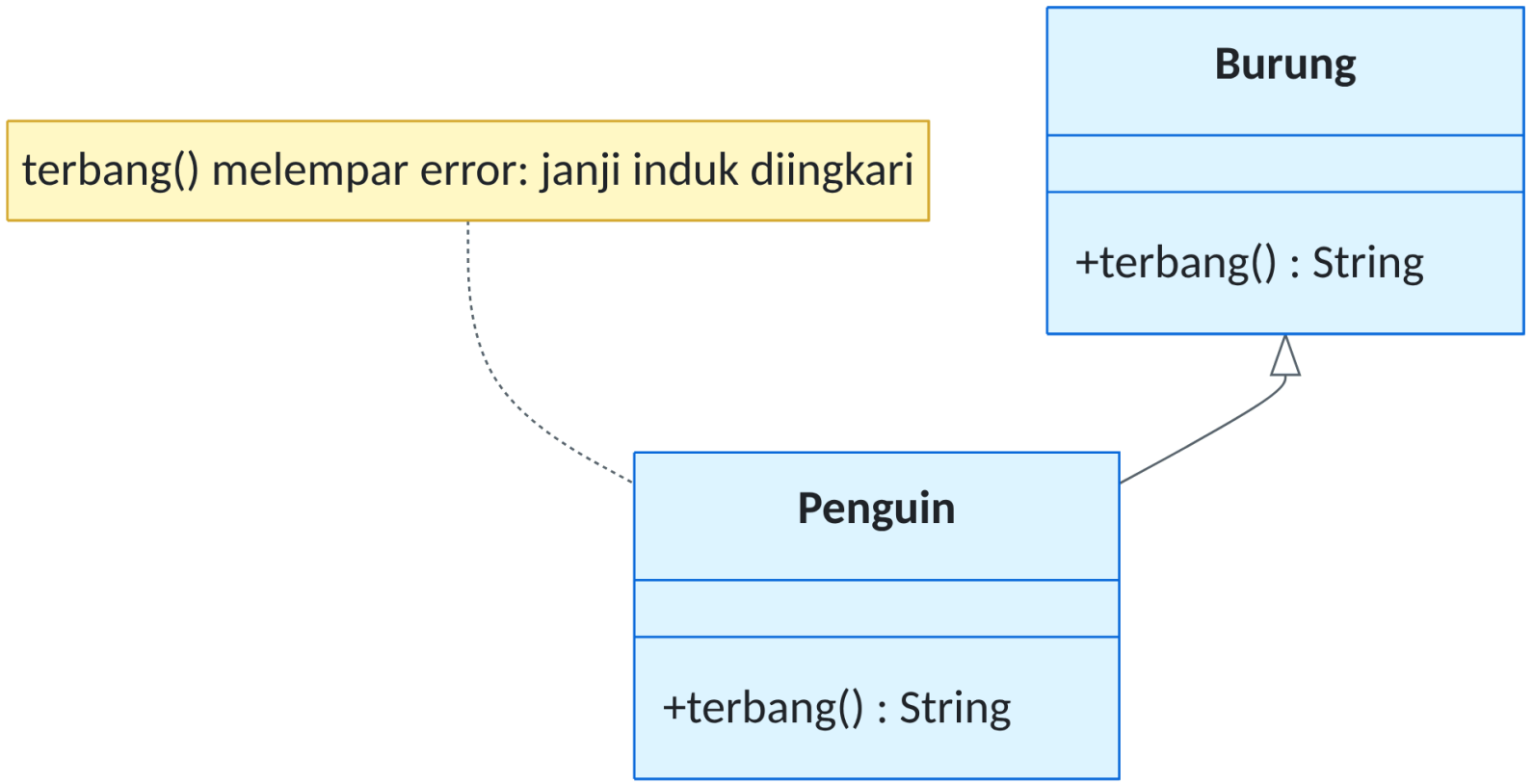
```python
class LayananKirim(ABC):
    # ... (dilewati)
    @abstractmethod
    def ongkir(self, berat_kg: int) -> int:
        # ... (dilewati)
# ... (dilewati)
class LayananSameDay(LayananKirim):
    # ... (dilewati)
    def ongkir(self, berat_kg: int) -> int:
        return 25000 + 5000 * berat_kg

class KalkulatorOngkir:
    # ... (dilewati)
    def hitung(self, layanan: LayananKirim, berat_kg) -> int:
        # ... (dilewati)
        return layanan.ongkir(berat_kg)
```

Java java/contoh/Contoh02Ocp.java

```java
interface LayananKirim {
    String nama();
    int ongkir(int beratKg);
}
// ... (dilewati)
class LayananSameDay implements LayananKirim {
    // ... (dilewati)
    public int ongkir(int beratKg) {
        return 25000 + 5000 * beratKg;
    }
}
class KalkulatorOngkir {
    public int hitung(LayananKirim layanan, int beratKg) {
        // ... (dilewati)
        return layanan.ongkir(beratKg);
    }
}
```

L: Kelas Anak Tidak Boleh Mengejutkan



Perbaikannya: janji dipecah. `BurungDasar` hanya berjanji bertelur, janji terbang pindah ke `BurungTerbang`, dan penguin cukup mewarisi `BurungDasar`.

Bentuk Kejutan yang Melanggar LSP

Bentuk kejutan	Contoh	Yang dilanggar
Melempar *exception* yang tidak dijanjikan	`Penguin.terbang()` melempar *error*	jaminan hasil diperlemah
Mengembalikan nilai di luar janji	kelas diskon mengembalikan potongan **negatif** sehingga total malah naik	jaminan hasil diperlemah
Meminta syarat lebih ketat dari induk	induk menerima jumlah porsi berapa pun, kelas anak hanya mau jumlah genap	prasyarat diperketat
Mengubah keadaan secara diam-diam	`Persegi` turunan `PersegiPanjang`: mengubah lebar ikut mengubah tinggi	jaminan hasil diperlemah

Kelas anak boleh meminta **lebih sedikit** dan memberi **lebih banyak** daripada induknya, tetapi tidak boleh sebaliknya.

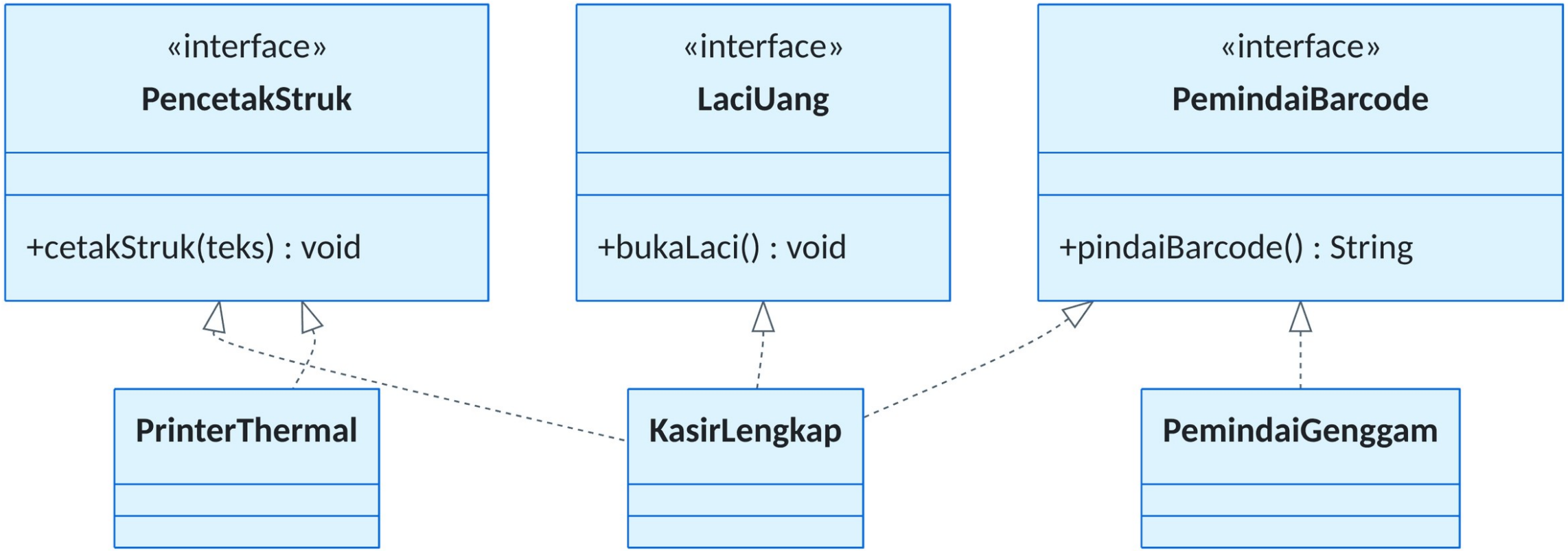
BAGIAN 4

4

I dan D: Kontrak Kecil dan Abstraksi

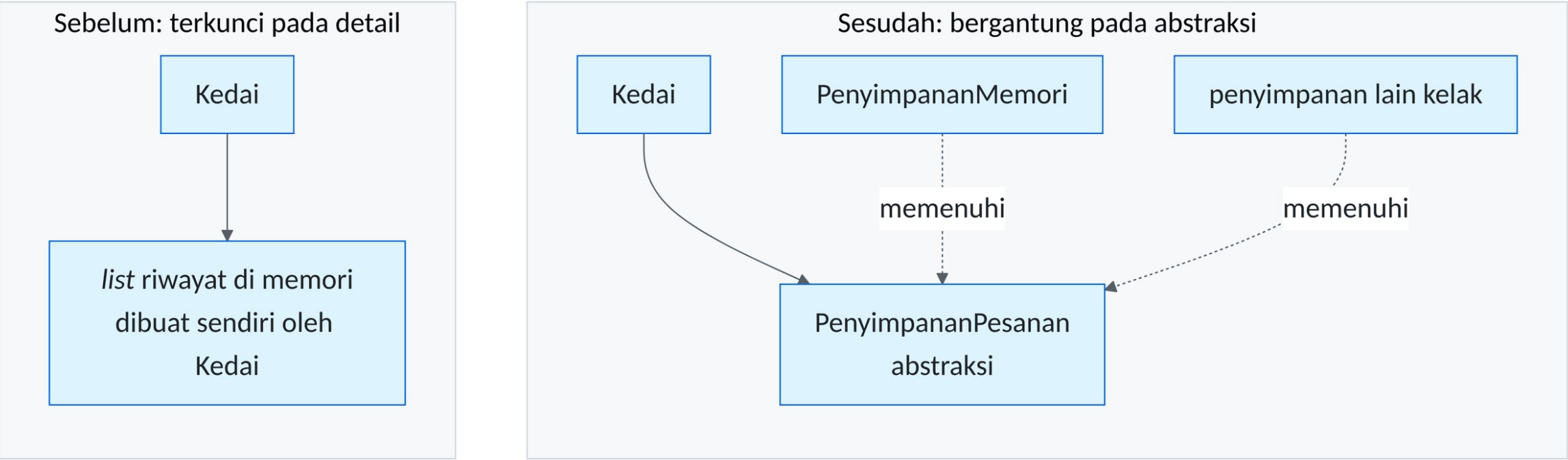
Kontrak yang pas dengan kebutuhan, dan kelas pengatur yang tidak terkunci pada detail teknis.

I: *Interface* Kecil yang Fokus



Kontrak gemuk `PerangkatKasir` dipecah tiga. Kasir lengkap memenuhi ketiganya, sedangkan *printer* termal cukup `PencetakStruk`.

D: Bergantung pada Abstraksi



Dulu detail berada di ujung panah. Kini detail (`PenyimpananMemori`) yang menunjuk ke abstraksi: itulah arti *inversion* (pembalikan).

Dependency Injection lewat Konstruktor

`PengingatKaku` membuat sendiri `SmsGateway`. `PengingatJadwal` menerima `Pengirim` dari luar lewat konstruktor.

Python python/contoh/contoh05_dip.py

```python
class PengingatKaku:
    def __init__(self):
        self._sms = SmsGateway()
    # ... (dilewati)
class Pengirim(ABC):
    """Abstraksi: apa pun yang bisa mengirim pesan."""
    @abstractmethod
    def kirim(self, tujuan: str, pesan: str):
        # ... (dilewati)
# ... (dilewati)
class PengingatJadwal:
    def __init__(self, pengirim: Pengirim):
        self._pengirim = pengirim
    # ... (dilewati)
```

Java java/contoh/Contoh05Dip.java

```java
class PengingatKaku {
    private SmsGateway sms;
    public PengingatKaku() {
        this.sms = new SmsGateway();
    }
    // ... (dilewati)
}
interface Pengirim {
    void kirim(String tujuan, String pesan);
}
class PengingatJadwal {
    private Pengirim pengirim;
    public PengingatJadwal(Pengirim pengirim) {
        this.pengirim = pengirim;
    }
    // ... (dilewati)
}
```

BAGIAN 5

5

Refactoring dan Prinsip Pendamping

Merapikan kode dengan aman, serta tiga prinsip pendek yang sering disebut pemrogram.

Refactoring: Ubah Struktur, Jaga Perilaku

Refactoring

Mengubah **struktur** kode **tanpa mengubah perilakunya**. *Output*, hasil perhitungan, dan reaksi program terhadap kesalahan harus tetap sama persis.

- Buktikan, jangan hanya merasa yakin: simpan *output* versi lama dan baru ke *file* dengan >, lalu bandingkan.
- `diff` (macOS/Linux) tidak mencetak apa pun jika sama; `fc` (Windows) menampilkan `FC: no differences encountered`.
- Aturan emas: langkah kecil, jangan dicampur dengan fitur baru atau perbaikan *bug*, dan uji banyak kasus.

DRY, KISS, YAGNI: Panduan, Bukan Aturan Mati

Prinsip	Artinya	Contoh pelanggaran
DRY (*Don't Repeat Yourself*)	satu aturan cukup ditulis di **satu tempat**	rumus denda perpustakaan disalin tiga kali untuk tiga jenis pustaka
KISS (*Keep It Simple, Stupid*)	pilih solusi **paling sederhana** yang benar	lima kelas dan tiga *interface* untuk menghitung luas satu persegi
YAGNI (*You Aren't Gonna Need It*)	jangan membuat sesuatu untuk kebutuhan yang **belum ada**	penyimpanan *database* dan *cloud*, padahal kedai baru butuh *list* di memori

Rancangan yang jauh lebih rumit daripada kebutuhannya disebut *over-engineering*. Tulis yang sederhana dan benar lebih dulu.



STUDI KASUS

Studi Kasus KedaiKita: Membongkar `KasirSerbaBisa`

Dua versi dengan *output* yang sama persis: `studi-kasus/sebelum` dan `studi-kasus/sesudah`.

Ke Mana Perginya Setiap Tugas?

Studi kasus KedaiKita

Tugas di `KasirSerbaBisa`	Sekarang dikerjakan oleh	Prinsip
Menyimpan data menu	`Kedai` bersama `Menu`, `Makanan`, `Minuman`	SRP
Menghitung subtotal	`Pesanan` dan `ItemPesanan`	SRP
Memilih dan menghitung diskon (rantai `if`)	`Diskon`: `TanpaDiskon`, `DiskonPersen`, `DiskonMember`	OCP, LSP
Menyusun dan mencetak struk	`StrukPesanan`	SRP
Menyimpan riwayat pesanan	`PenyimpananPesanan` (wujudnya `PenyimpananMemori`), disuntikkan ke `Kedai`	DIP, ISP
Membuat nomor pesanan P001, P002, …	`Pesanan` (atribut milik kelas)	SRP

Nama dan poin pelanggan yang dulu dikirim sebagai parameter lepas kini menjadi objek `Pelanggan` (enkapsulasi).

Sebelum: Satu *Method* Mengurus Semuanya

Studi kasus KedaiKita

Potongan `proses_pesanan()`/`prosesPesanan()`: data menu, subtotal, diskon, struk, dan riwayat bercampur di satu tempat.

Python python/studi-kasus/sebelum/kasir_serba_bisa.py

```python
for kode, jumlah in daftar_beli:
    nama, harga = self.menu[kode]
    subtotal += harga * jumlah
# ... (dilewati)
if jenis_diskon == "member":
    # ... (dilewati)
elif jenis_diskon == "promo":
    potongan = subtotal * 15 // 100
# ... (dilewati)
print("KEDAIKITA - STRUK PESANAN")
# ... (dilewati)
# Tugas 4: menyimpan riwayat (hanya bisa ke list di memori)
self.riwayat.append((nomor, nama_pelanggan, total))
```

Java java/studi-kasus/sebelum/KasirSerbaBisa.java

```java
for (ItemBeli beli : daftarBeli) {
    DataMenu data = menu.get(beli.kode);
    subtotal += data.harga * beli.jumlah;
}
// ... (dilewati)
if (jenisDiskon.equals("member")) {
    // ... (dilewati)
} else if (jenisDiskon.equals("promo")) {
    potongan = subtotal * 15 / 100;
    // ... (dilewati)
} else {
    // ... (dilewati)
}
// ... (dilewati)
System.out.println("KEDAIKITA - STRUK PESANAN");
// ... (dilewati)
// Tugas 4: menyimpan riwayat (hanya bisa ke list di memori)
riwayat.add(new CatatanRiwayat(nomor, namaPelanggan, total));
```

Sesudah: Setiap Tugas Punya Rumah Sendiri

Studi kasus KedaiKita

Potongan program utama versi sesudah. Struk Siti dengan diskon member tetap sama persis seperti versi sebelum.

Python python/studi-kasus/sesudah/main.py

```python
kedai = Kedai("KedaiKita", penyimpanan)
# ... (dilewati)
pesanan = kedai.buat_pesanan(pelanggan)
# ... (dilewati)
pesanan.atur_diskon(diskon)
StrukPesanan(pesanan).cetak()
```

Java java/studi-kasus/sesudah/Main.java

```java
Kedai kedai = new Kedai("KedaiKita", penyimpanan);
// ... (dilewati)
Pesanan pesanan = kedai.buatPesanan(pelanggan);
// ... (dilewati)
pesanan.aturDiskon(diskon);
new StrukPesanan(pesanan).cetak();
```

Output

```
No. Pesanan : P002
Pelanggan   : Siti Aminah
...
Subtotal                        Rp31.000
Diskon Member 10%               -Rp3.100
TOTAL                           Rp27.900
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Kode yang melanggar SOLID pasti salah.”	Belum tentu. `KasirSerbaBisa` menghasilkan *output* yang benar. SOLID menilai **kemudahan diubah**, bukan kebenaran hasil hari ini.
“SRP berarti satu kelas hanya boleh punya satu *method*.”	SRP berarti satu **alasan untuk berubah**. `PenghitungNilai` punya dua *method*, tetapi keduanya melayani satu urusan: aturan penilaian.
“DIP sama saja dengan *dependency injection*.”	DIP adalah prinsip arah ketergantungan; *dependency injection* hanya salah satu tekniknya. Menyuntikkan objek bertipe kelas konkret belum memenuhi DIP.

Kuis Cepat

Kontrak `PerangkatKasir` mewajibkan `cetak_struk()`, `pindai_barcode()`, dan `buka_laci()`. Printer Bluetooth yang hanya bisa mencetak terpaksa mengisi dua *method* lain dengan melempar *error*. Perbaikan yang paling tepat adalah …

A Memecah kontrak menjadi kontrak kecil seperti `PencetakStruk`, `PemindaiBarcode`, dan `LaciUang`

B Mengisi kedua *method* itu dengan isi kosong agar tidak ada *error*

C Menambahkan lebih banyak *method* ke `PerangkatKasir` agar lengkap

D Menghapus semua kontrak dan memeriksa jenis perangkat dengan percabangan

Kuis Cepat: Jawaban

Kontrak `PerangkatKasir` mewajibkan `cetak_struk()`, `pindai_barcode()`, dan `buka_laci()`. Printer Bluetooth yang hanya bisa mencetak terpaksa mengisi dua *method* lain dengan melempar *error*. Perbaikan yang paling tepat adalah ...

✓ Memecah kontrak menjadi kontrak kecil seperti `PencetakStruk`, `PemindaiBarcode`, dan `LaciUang`

B Mengisi kedua *method* itu dengan isi kosong agar tidak ada *error*

C Menambahkan lebih banyak *method* ke `PerangkatKasir` agar lengkap

D Menghapus semua kontrak dan memeriksa jenis perangkat dengan percabangan

Jawaban: A

ISP meminta *interface* kecil yang fokus sehingga setiap perangkat hanya berjanji sebanyak kemampuannya. Pilihan B menyembunyikan masalah: pemanggil mengira pemindaian berhasil padahal tidak terjadi apa-apa (tetap melanggar LSP). Pilihan C memperparah kontrak gemuk, dan pilihan D kembali ke rantai percabangan yang melanggar OCP.

Kuis Cepat

Di KedaiKita versi sesudah *refactoring*, pemilik ingin struk menampilkan nomor WhatsApp kedai. Kelas mana yang paling tepat diubah?

A `Pesanan`

B `StrukPesanan`

C `Diskon`

D `Kedai`

Kuis Cepat: Jawaban

Di KedaiKita versi sesudah *refactoring*, pemilik ingin struk menampilkan nomor WhatsApp kedai. Kelas mana yang paling tepat diubah?

A `Pesanan`

✓ `StrukPesanan`

C `Diskon`

D `Kedai`

Jawaban: B

Menyusun dan mencetak struk adalah satu-satunya urusan `StrukPesanan` (SRP). `Pesanan` hanya mengurus item dan perhitungan, `Diskon` hanya mengurus potongan, dan `Kedai` mengurus menu serta pembuatan pesanan. Inilah manfaat SRP: kamu langsung tahu harus membuka *file* yang mana.

Ringkasan

- Program yang benar belum tentu mudah diubah. **Prinsip desain** membantu menyusun kelas yang tahan perubahan.
- ***Code smell***: kelas serba bisa, rantai percabangan jenis, *override* yang menolak, *interface* gemuk, kode kembar.
- **SRP**: satu kelas, satu alasan untuk berubah. **OCP**: perilaku baru lewat kelas baru, bukan membongkar kode lama.
- **LSP**: kelas anak menepati janji induknya, tidak memperketat prasyarat dan tidak memperlemah jaminan hasil.
- **ISP**: *interface* kecil yang fokus. **DIP**: pengatur dan detail sama-sama bergantung pada abstraksi.
- ***Refactoring*** mengubah struktur tanpa mengubah perilaku. Buktikan dengan `diff` atau `fc`.
- DRY, KISS, dan YAGNI melengkapi SOLID. Semuanya panduan, bukan aturan mati.

Langkah Berikutnya

- Jalankan contoh 01—06, lalu studi kasus versi sebelum dan sesudah di jalur pilihanmu.
- Buktikan *output* kedua versi sama dengan `diff` atau `fc`, lalu kerjakan latihan dan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-13-Prinsip-SOLID